

Operating Systems Project Report
Name: Hadi Muselmani
Course: CMPS442 Operating Systems
Project: CPU Scheduling and Bankers Algorithm
ID:202304319

Part A :

Project number 3 : “FCFS” Scheduling algorithm

Introduction

This project involves simulating CPU scheduling using the First Come First Serve (FCFS) algorithm in the OS Simulator. The system includes 12 processes with different arrival times, CPU bursts, and I/O bursts generated within given ranges.

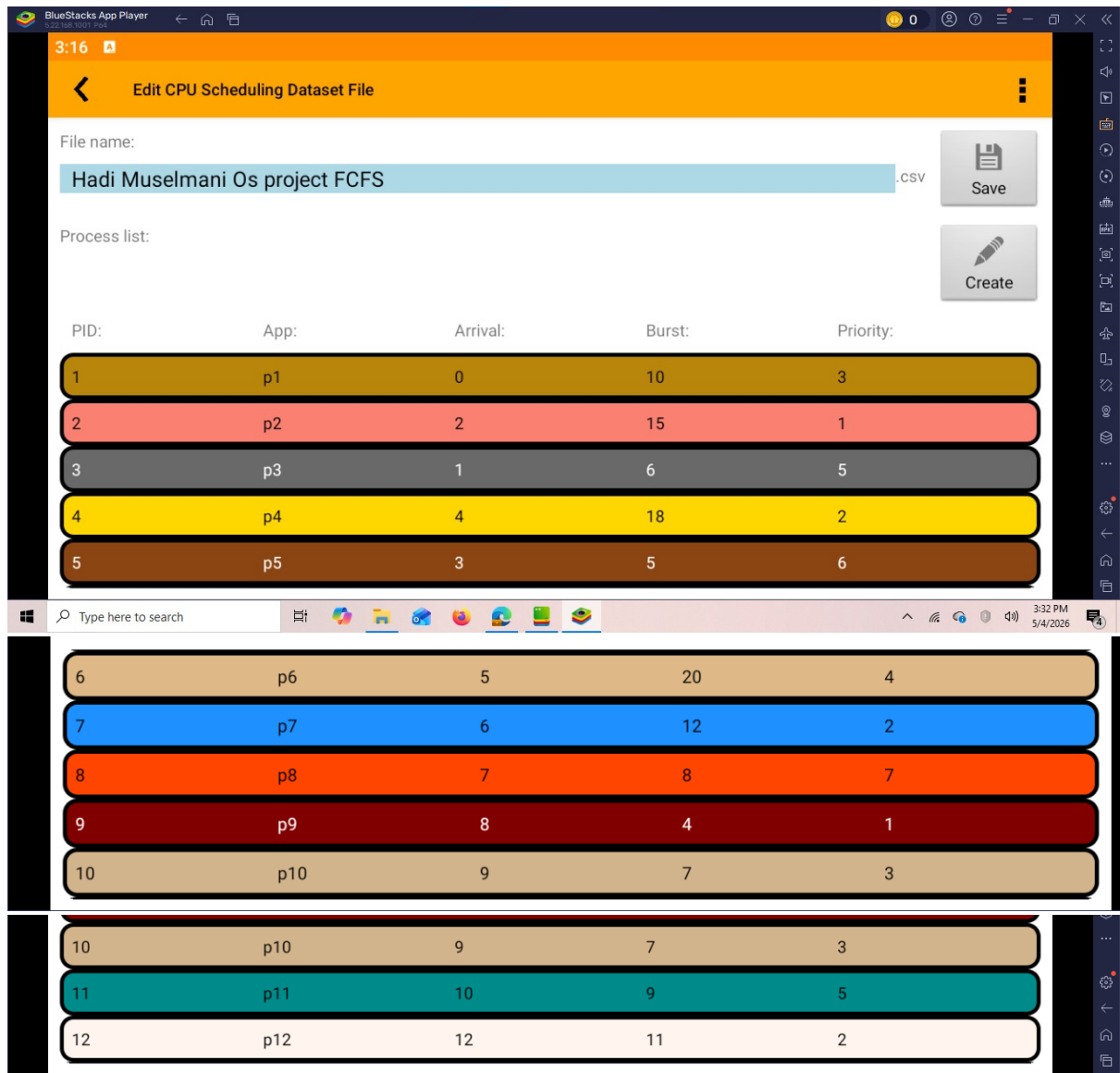
The aim is to understand how FCFS schedules processes based on arrival order and to analyze system performance, including waiting time and process execution behavior.

FCFS Scheduling : given

Parameter	Value
Number of Processes	12
Degree of Multiprogramming (DOM)	20
Number of Bursts	15
CPU Burst Range	5 – 20
I/O Burst Range	20 – 100
Priority Range	1 – 7
Arrival Time Range	0 – 25
Context switch	2
Alpha	0.5
tau	50

I downloaded BlueStacks and opened it. Inside BlueStacks, I went to the Google Play Store and downloaded the OS Simulator Algorithm app. After launching the app, I selected the CPU Scheduler because it is required for my Project Number 3.

This is my project configuration before starting my FCFS simulation :



I added 3 screenshots for the initial configuration because it couldnt fit in one screenshot .

These processes were created manually in the simulator each process was a given an id,app time,Arrival time ,burst value, and priority . The table below shows my dataset .

PID	App	Arrival Time	Burst Time	Priority
1	p1	0	10	3
2	p2	2	15	1
3	p3	1	6	5
4	p4	4	18	2
5	p5	3	5	6
6	p6	5	20	4
7	p7	6	12	2
8	p8	7	8	7
9	p9	8	4	1
10	p10	9	7	3
11	p11	10	9	5

12	p12	12	11	2
----	-----	----	----	---

Experimentation Steps

I opened the OS Simulator application using BlueStacks.

I selected the CPU Scheduling section from the main menu.

I chose the First Come First Serve (FCFS) scheduling algorithm.

I configured the simulation with 12 processes.

I set the required parameters, including arrival time range (0–25), CPU burst range (5–20), I/O burst range (20–100), and other given settings.

I generated the process dataset automatically in the simulator.

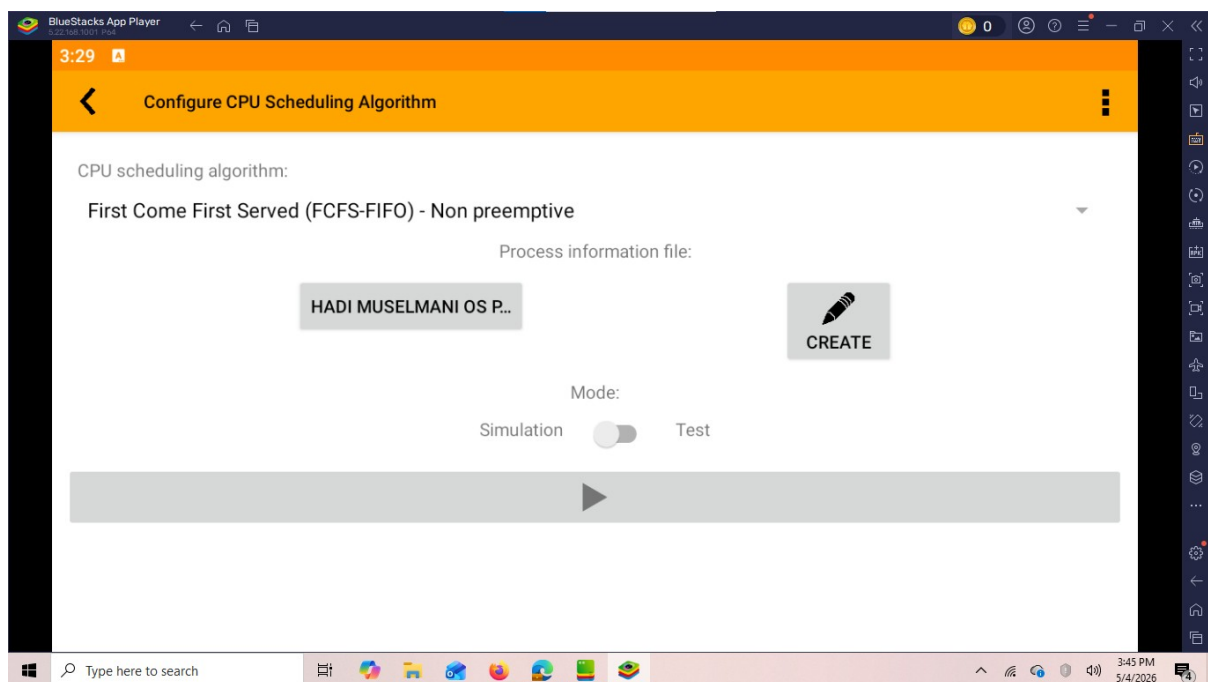
I reviewed the created processes with their arrival times, burst times, and other attributes.

I started the simulation using the play/step button.

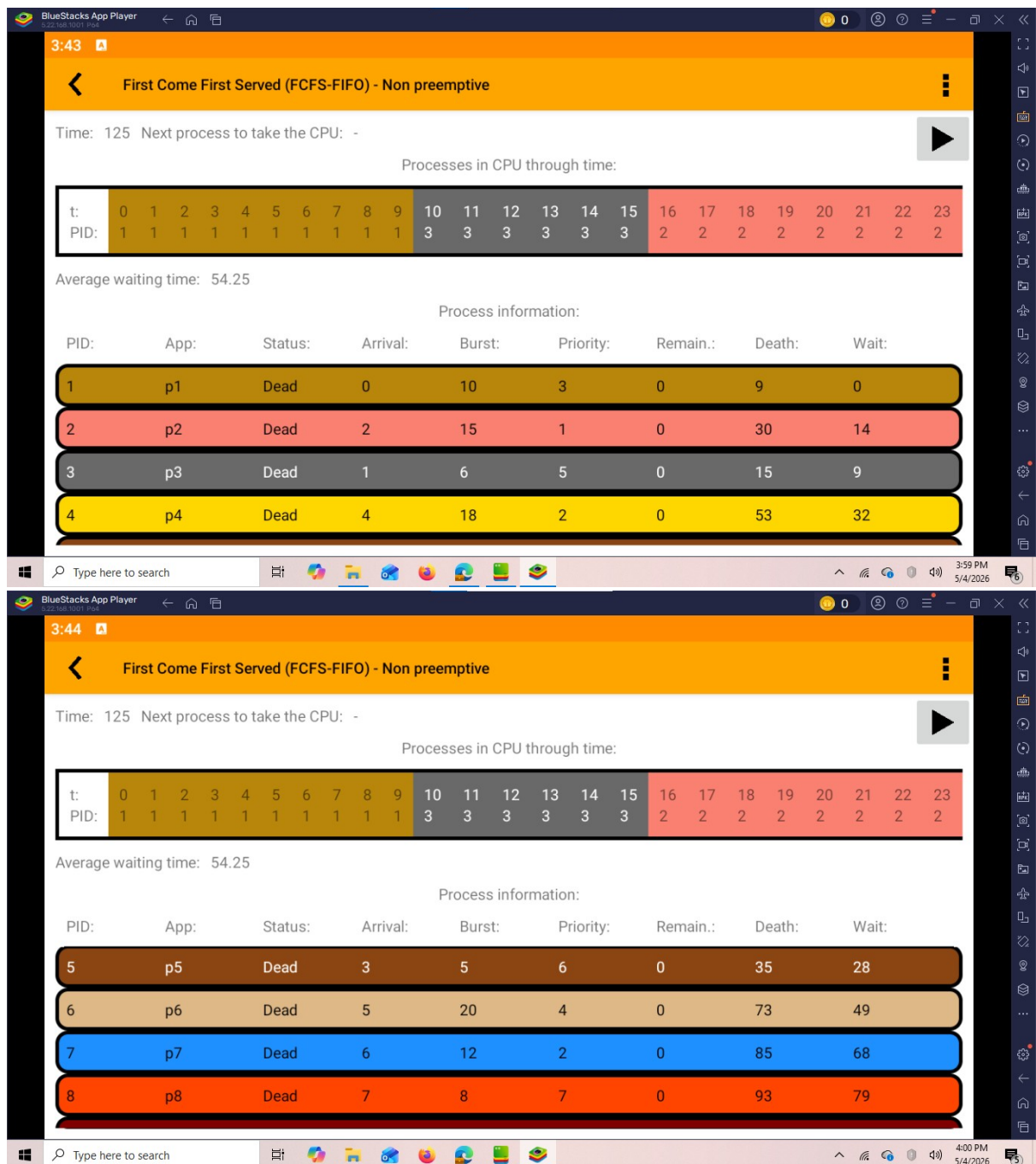
I observed the execution order of processes based on FCFS scheduling.

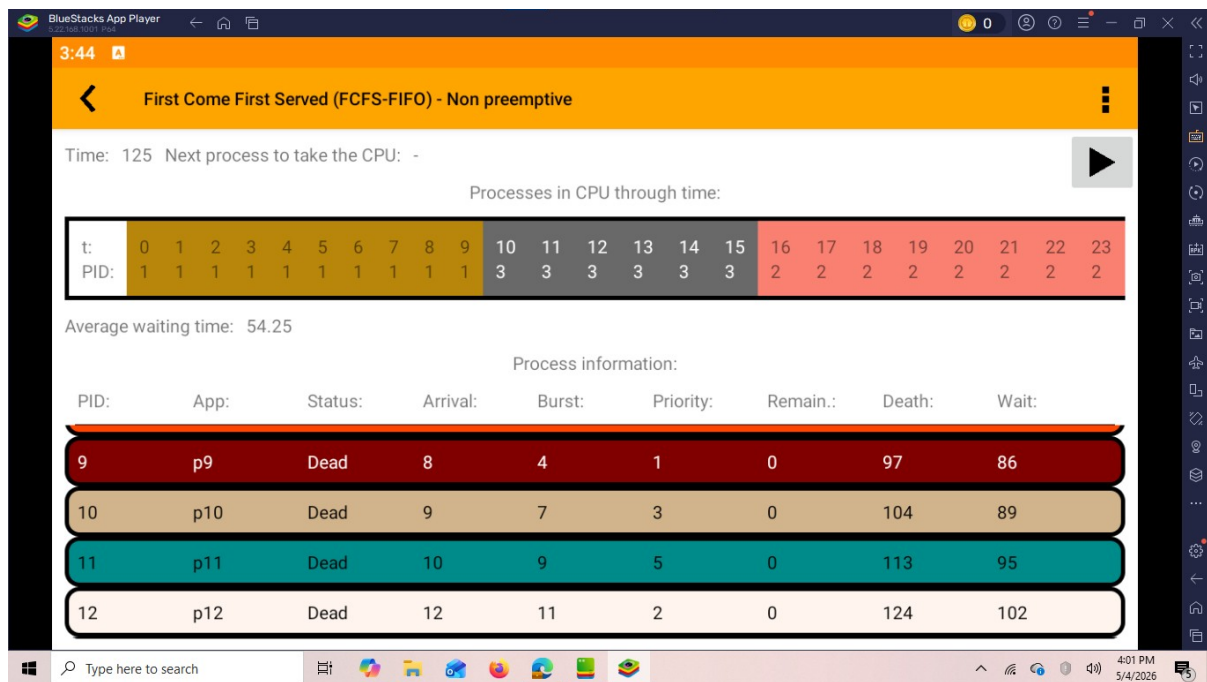
I recorded the waiting time, turnaround time, and completion results.

I took screenshots of the initial configuration, execution process, and final output.



Result : After running the FCFS simulation , The simulator showed the execution order of the processes , completion time,waiting time and Average time was 54.25.





Output Table of the FCFS Simulation :

Proces s	Arrival Time	Priority	Waiting Time	Completion Time
P1	0	3	0	9
P2	2	1	14	30
P3	1	5	9	15
P4	4	2	32	53
P5	3	6	28	35
P6	5	4	49	73
P7	6	2	68	85
P8	7	7	79	93
P9	8	1	85	97
P10	9	3	92	104
P11	10	5	101	113
P12	12	2	102	124

FCFS Scheduling Diagram (Gantt Chart)

Time Interval	Process
0 – 10	P1
10 – 16	P3
16 – 31	P2
31 – 36	P5

36 – 54	P4
54 – 74	P6
74 – 86	P7
86 – 94	P8
94 – 98	P9
98 – 105	P10
105 – 114	P11
114 – 125	P12

Discussion :

In this project, I used the FCFS (First Come First Serve) scheduling algorithm in the OS Simulator to observe how processes are executed according to their arrival times. The simulation showed that processes are handled in the exact order they arrive in the ready queue without interruption.

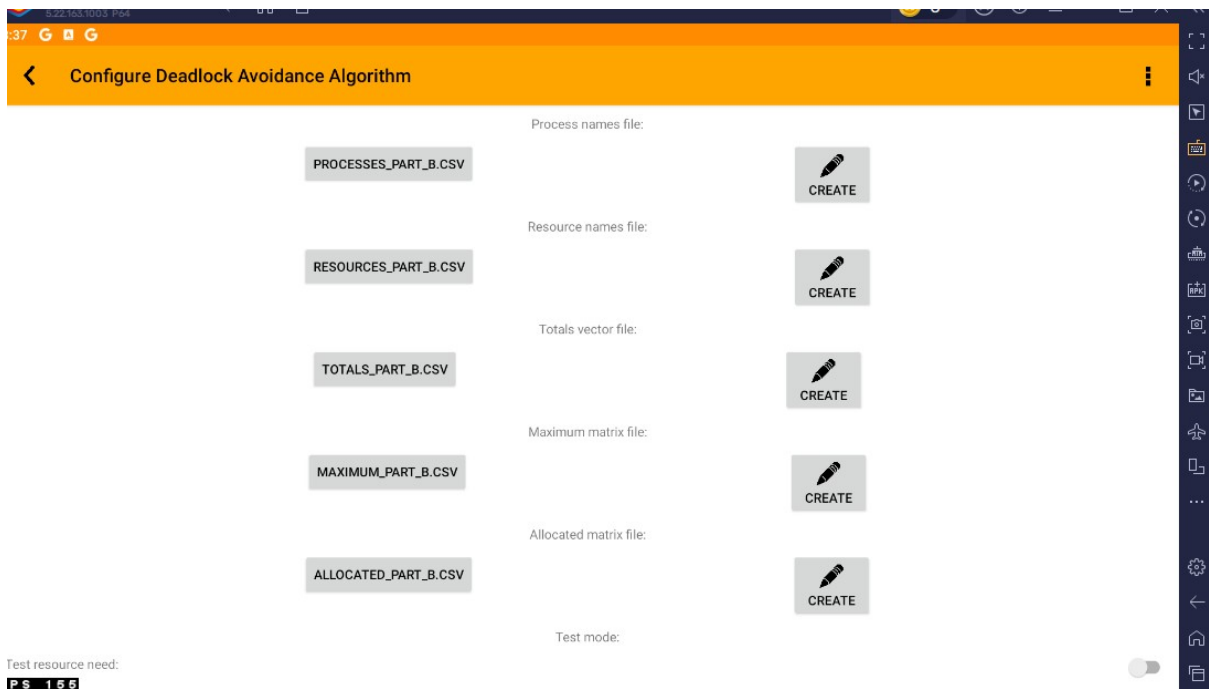
During the execution, I observed the waiting time and completion time of each process. Some processes experienced long waiting times because FCFS does not consider process priority or burst time. I also learned how context switching and arrival times affect overall CPU performance.

Overall, the simulation helped me better understand CPU scheduling, process execution order, and how operating systems manage multiple processes step by step.

Thank you

Part B: Bankers Algorithm

Part B is a required section of the project. It applies the Banker's Algorithm to determine whether the system is in a safe state. The system consists of four processes and three types of resources. To perform the analysis, the algorithm uses the Max Matrix, Allocation Matrix, Available Vector, and calculates the Need Matrix. Based on these values, it checks if a safe sequence of process execution exists.



This is the Initial Banker's Algorithm configuration before running the simulation.

Given Data

The project gives the Max Matrix, Allocation Matrix, and Available Vector. These values were used to calculate the Need Matrix and to check whether the system is safe.

MAX MATRIX

Process	R1	R2	R3
P1	7	5	3
P2	3	2	2
P3	9	0	2
P4	2	2	2

ALLOCATION MATRIX

Process	R1	R2	R3
P1	0	1	0
P2	2	0	0
P3	3	0	2
P4	2	1	1

Available Vector

Resource	Available
R1	3
R2	3
R3	4

In the simulator, I entered the total vector instead of the available vector. The total resources were calculated by adding the allocated resources to the available resources.

Resource	Total
R1	10
R2	5
R3	7

3.2 Need Matrix

The Need Matrix is calculated using the formula: $\text{Need} = \text{Max} - \text{Allocation}$. This matrix shows how many more resources each process may still need before it can finish execution

Process	R1	R2	R3
P1	7	4	3
P2	1	2	2
P3	6	0	0
P4	0	1	1

3:42 Simulate Deadlock Avoidance Algorithm

Tap on any process to perform any action on it (request resources, kill it, etc.)

Process information:

PID:	1	App:	P1	Status:	Ready
Resource	Maximum	Allocated	Need		
R1	7	0	7		
R2	5	1	4		
R3	3	0	3		

PID:	2	App:	P2	Status:	Ready
Resource	Maximum	Allocated	Need		
R1	3	2	1		
R2	2	0	2		
R3	2	0	2		

PID:	3	App:	P3	Status:	Ready
Resource	Maximum	Allocated	Need		
R1	9	3	6		
R2	0	0	0		
R3	2	2	0		

Resources

Resource	Available	Total
R1	3	10
R2	3	5
R3	4	7

P S 2 1 6

Banker's Algorithm state showing P1, P2, Need values, and Available resources.

3:43 Simulate Deadlock Avoidance Algorithm

Tap on any process to perform any action on it (request resources, kill it, etc.)

Process information:

PID:	3	App:	P3	Status:	Ready
Resource	Maximum	Allocated	Need		
R1	9	3	6		
R2	0	0	0		
R3	2	2	0		

PID:	4	App:	P4	Status:	Ready
Resource	Maximum	Allocated	Need		
R1	2	2	0		
R2	2	1	1		
R3	2	1	1		

Resources

Resource	Available	Total
R1	3	10
R2	3	5
R3	4	7

P S 1 7 8

Initial Banker's Algorithm state showing P3 and P4.

Safe Sequence

To check if the system is safe, I compared the Need values of the processes with the Available Vector. At the beginning, the Available Vector was $R1 = 3$, $R2 = 3$, and $R3 = 4$. A process can execute if its Need values are less than or equal to the Available values.

Step	Available Before	Process Selected	Need	Available After
1	(3, 3, 4)	P2	(1, 2, 2)	(5, 3, 4)
2	(5, 3, 4)	P4	(0, 1, 1)	(7, 4, 5)
3	(7, 4, 5)	P1	(7, 4, 3)	(7, 5, 5)
4	(7, 5, 5)	P3	(6, 0, 0)	(10, 5, 7)

Therefore, the safe sequence is: P2 → P4 → P1 → P3. All can finish in this order.

Results

Your final score is 4/4 (100%)

Question 1		✓
Process in position 1 in the safe sequence would be...		
Your answer	Correct answer	
2 (P2)	2 (P2)	

Question 2		✓
Process in position 2 in the safe sequence would be...		
Your answer	Correct answer	
4 (P4)	4 (P4)	

Question 3		✓
Process in position 3 in the safe sequence would be...		
Your answer	Correct answer	
1 (P1)	1 (P1)	

Question 4		✓
Process in position 4 in the safe sequence would be...		
Your answer	Correct answer	
3 (P3)	3 (P3)	

ACCEPT

Final safe sequence result in the Banker's Algorithm simulator.

Scenario: Killing a Process

The simulator also allows actions such as killing a process. I tested this by killing process P3. After killing P3, the status of P3 changed to Dead. This means the process was removed from execution, and the system state changed.

PID:	3	App:	P3	Status:	Dead
Resource	Maximum	Allocated	Need		

System state after killing process P3.

This scenario showed that removing a process changes the process table and affects the current system state. Since P3 was no longer active, it no longer needed to request more resources.

Discussion for Part B

While working with the Banker's Algorithm in the simulator, I understood how the system decides whether allocating resources is safe or not. The simulator checks different conditions before allowing a process to proceed. The Need Matrix plays an important role because it shows how many more resources each process still requires to finish execution.

In this case, the system was in a safe state because there was at least one order in which all processes could complete without causing a deadlock.

The safe sequence obtained was **P2 → P4 → P1 → P3**. This order is logical because P2 required fewer resources compared to what was available, so it could execute first. Once P2 finished, it released its allocated resources, which increased the available resources in the system. This made it possible for P4 to execute next, followed by P1 and finally P3.

When testing the kill process scenario, I observed how the simulator updates the system dynamically. For example, when P3 was terminated, its state changed to dead, and its allocated resources were released back to the system. This helped me understand how changes in processes can impact resource allocation and deadlock avoidance.

Conclusion

The safe sequence obtained was **P2 → P4 → P1 → P3**. This order is logical because P2 required fewer resources compared to what was available, so it could execute first. Once P2 finished, it released its allocated resources, which increased the available resources in the system. This made it possible for P4 to execute next, followed by P1 and finally P3.